

The German Tank Problem — Instructor Notes

Class 11 · estimators and sampling distributions · handout + skeleton notebook

The idea. You can't judge an estimator by the one answer it gives you, because you get one capture and you never learn the truth. You judge it by what it *would* say across every capture that could have happened. Students build that picture by simulating a fleet whose size they already know. The obvious rule (report the largest serial) turns out to be wrong in the same direction every time. The corrected rule is right on average, and on any one capture it can still miss by a lot.

What students produce. A committed rule and estimate, two tested functions, two sampling distributions, a ten-number comparison table, and a simulated tail probability for the 1500-tank claim. **What this does not claim to teach:** unbiasedness or minimum variance as formal ideas, maximum likelihood, or hypothesis testing by name. Part 5 is a hypothesis test with the word left out on purpose (the notebook says so in its last cell).

Materials. Handout, skeleton notebook on the hub. Board space to list the Part 1 rules. The true fleet size for the end of class (see *Before You Leave*).

Timing (50 min)

Segment	Min	What happens
Part 1	7	Background is pre-reading. Teams commit to a rule and a number in ink, notebook closed. List every team's rule and estimate on the board.
The correction	5	Walk the number-line figure. Do the two sanity checks ($k = N$, $k = 1$) out loud.
Part 2	10	Draws, write <code>est_corrected</code> , pass the 14 / 16.5 test, apply to the real serials, 2.4 by hand.
Part 3	10	Both loops at 10, then 1000. Histograms, 3.1–3.3.
Part 4	8	The table, 4.1, 4.2.
Part 5	7	Simulate the 1500 claim, 5.1, 5.2.
Before You Leave	3	Announce the true fleet size; one or two teams say whether the method was wrong.

Watch out. The 50-minute budget is an estimate. If you're short, cut 4.2 to a show of hands and send E1 and E2 home. Don't cut Part 5 or the reveal; the reveal only works after Part 3.

The numbers

For a fleet of $N = 1000$ and captures of $k = 5$, the largest serial M has an exact distribution, $P(M = m) = \binom{m-1}{4} / \binom{1000}{5}$, so every row below except the "class range" column is exact, not simulated. The corrected estimator is $1.2M - 1$.

	mean	median	SD	IQR	within 20%
max (exact)	834.2	871	140.6	186	0.675
class range, 90%	827–842	863–880	134–147	172–198	0.65–0.70
corrected (exact)	1000.0	1044	168.7	223	0.869
class range, 90%	991–1009	1034–1055	161–176	207–238	0.85–0.89

“Class range” is the middle 90% of what one team’s 1000-capture simulation gives (2000 simulated teams). A team value outside it usually means a bug; see the watch-outs in Part 3.

Other numbers you’ll want:

- The real capture (62, 214, 389, 605, 880): est_max 880, est_corrected **1055.0**, twice the mean minus one **859**.
- The bias of max is $N - k(N + 1)/(k + 1) = 1000 - 834.2 \approx 166$, about one average gap, $(N + 1)/(k + 1)$. That’s exactly what the correction adds back.
- $P(M = 1000) = 5/1000 = 0.5\%$: the only way max is ever right.
- The corrected rule overshoots 1000 **59.7%** of the time, even though its average is 1000.
- Root-mean-square error: max 217 (bias and spread combined), corrected 169.
- Part 5: $P(M \leq 880 \mid N = 1500) = \binom{880}{5} / \binom{1500}{5} = \mathbf{0.069}$. Class range 0.058–0.081.

Part 1. Commit First

(≈ 7 min)

1.1 Any rule that anyone could apply unambiguously. Put every rule on the board. The ones you’re likely to see, with what they give for this capture:

Rule	Estimate	Error if $N = 897$
largest serial	880	–17
twice the mean, minus 1	859	–38
twice the median	778	–119
largest + smallest – 1	941	+44
largest + one average gap – 1	1055	+158
“round up”: 900 or 1000	900 / 1000	+3 / +103

$N = 897$ is the suggested true value; see *Before You Leave*. “Largest + smallest” is a good student rule: it assumes the gap above the largest matches the gap below the smallest. It’s unbiased too, just noisier than the corrected rule (SD about 217 vs. 169) because it uses one gap instead of the average of five.

Note. Resist ranking the rules now. The whole activity is about building a way to compare them that doesn’t depend on knowing the answer. Leave the list up; you’ll come back to it at the end.

1.2 Every captured serial is one of the tanks that exists, so none can exceed N , and neither can the largest. The rule can equal N but never overshoot it; it’s right only if you happened to capture the very last tank built (5 chances in 1000 here). Any miss is on the low side, so on average it must be low. Push for “can never be too high,” not “is probably too low.”

Where the Correction Comes From

(≈ 5 min)

Draw the number line. The claim students should leave with: *the gaps between random serials are exchangeable, so the unseen gap above the maximum is estimated by the average of the seen ones*. The –1

is a discreteness correction and not worth more than a sentence. If someone asks, the result $m(1 + 1/k) - 1$ is the standard textbook estimator for this problem: unbiased, and with the smallest spread of any unbiased rule. Don't claim more than that about how the wartime analysts computed their numbers.

Part 2. One Capture

(≈ 10 min)

2.1 Varies. Half of all draws land between 759 and 945 (median 871); about one draw in six is 700 or below. If the whole room sees only values in the 900s, someone is sampling from the wrong array.

2.2 14 and 16.5. The usual failures: `max(sample) * 1 + 1/len(sample) - 1` (precedence, returns 13.25); forgetting the `-1` (17.5); hard-coding 5 for k (the test has four serials, so it gives 15.8). The test exists to catch that last one; hard-coded 5 still works everywhere else in the notebook, which is why it's dangerous.

2.3 `est_max 880, est_corrected 1055.0. captured = make_array(62, 214, 389, 605, 880).`

2.4 $2 \times 430 - 1 = 859$. It can't be the true number: tank 880 exists, so $N \geq 880$. The rule is still unbiased: $E[\bar{x}] = (N + 1)/2$, so $2\bar{x} - 1$ averages exactly N . Over many captures it returns a number below the sample's own largest serial about **20%** of the time, and its SD is about 258, against 169 for the corrected rule. The takeaway: "right on average" is a property of the *procedure* over many captures; "sensible" is a property of *this* answer. A rule can have the first without the second. This is the seed for E2.

Part 3. A Thousand Parallel Wars

(≈ 10 min)

3.1 Both are piled up at the right with a long tail to the left. `max` peaks in the 950–1000 bin and stops at 1000; `corrected` is the same shape stretched by 1.2, peaking in the 1150–1200 bin, and it can't exceed 1199.

Watch out. The sliver past 1000. About 5 of 1000 captures include tank 1000, and the histogram's bins are closed on the left, so those land in the **1000–1050** bin. It looks like `max` exceeded 1000. It didn't; the bar is the captures where it was exactly right. Worth pointing at: it's the rule succeeding.

3.2 `corrected`, in the balance-point sense: its mean is 1000. Its peak and median are *above* 1000 (median about 1044), and it overshoots 60% of the time. Both are skewed left (long tail toward small estimates).

Watch out. Students who read "centered" as "where the peak is" will say neither histogram is centered on 1000, and they have a point. Accept it, then ask where the histogram would balance on a fingertip. Mean vs. median vs. mode on a skewed shape is a Part 4 idea arriving early.

3.3 From taking the maximum. The largest of five draws is pushed toward the top: it's at most x only if all five are, so $P(M \leq x) \approx (x/N)^5$. Values pile up just under the ceiling and thin out slowly below. There's a hard wall at 1000 and no wall on the low side. The corrected rule is a straight-line rescaling of `max` ($1.2M - 1$), so it inherits the skew. The correction moves the balance point; it doesn't change the shape. Strong answers name both the ceiling and "one of five draws is the biggest."

Watch out. Common simulation bugs, by symptom.

- Histogram is a single spike, SD = 0: the loop reuses sample from an earlier cell instead of drawing a new one inside the loop.
- Column length mismatch from `Table().with_columns`: one loop is still at 10 and the other at 1000.
- Only 10 values in a histogram: the 10 → 1000 change was never made (or was made but the cell wasn't rerun).
- corrected mean near 834 instead of 1000: the second loop appends `est_max`.

Computing `corrected_estimates = max_estimates * 1.2 - 1` from the first loop is correct and fine. It reuses the same captures, which is what 3.3's "same shape stretched" is about.

Part 4. Which Estimator Would You Take Into a War?

(≈ 8 min)

Table values: see *The numbers*. In the one run used to check the key (seed 1942): max 840.2 / 875.5 / 137.2 / 177.0 / 0.690, corrected 991.1 / 1035.8 / 177.6 / 225.0 / 0.863.

Watch out. & needs parentheses around each comparison. Without them, `max_estimates >= 800 & max_estimates <= 1200` raises a `TypeError` about `bitwise_and`; with and instead of &, it's the "truth value of an array ... is ambiguous" `ValueError`. The notebook mentions both rules, but the `bitwise_and` message doesn't point students back to it.

4.1 max, always low, by about 166 on average (mean about 834). It can't come out any other way: no captured serial exceeds N (1.2), so max is at most 1000 on every capture and less than 1000 on all but 0.5% of them. An average of numbers that are all ≤ 1000 , nearly all strictly less, has to be below 1000. Bonus: the shortfall is about one average gap, $1001/6 \approx 167$, which is exactly what the correction adds back.

4.2 Either is defensible if argued. The SD is pulled up by the long low tail; the IQR describes the middle half and ignores the tail. The better answer notices *which* tail is long: the tail is on the **underestimate** side, the dangerous side for a general. So a single spread number, of either kind, undersells the risk that matters. A general would do better with "half the time between 910 and 1133, and 1 time in 4 below 900." That's percentiles, and it's where Class 19 goes.

Note. The trade-off to surface. The corrected rule has the *larger* SD (169 vs. 141; stretching by 1.2 stretches the spread too) and still lands within 20% far more often (87% vs. 68%) because it is aimed at the right place. Removing bias cost some spread, and it was worth it. RMSE puts both in one number: 217 vs. 169.

Part 5. Testing a Claim

(≈ 7 min)

5.1 About 0.07 (exact 0.069; a team's 1000 simulations give 0.058–0.081).

Watch out. The 0.5 tell. If a team reports about 0.5, their loop draws from `serials`, which is still 1 to 1000. The sample must come from `np.arange(1, claimed + 1)`. (With a fleet of 1000, the largest of five is ≤ 880 about 53% of the time.)

5.2 Model answer: “If the enemy really had 1500 tanks, a capture of five with nothing numbered above 880 would happen only about 7 times in 100. That’s unusual, but it isn’t rare enough to rule 1500 out. Our capture is evidence against the 1500 figure, not proof that it’s wrong.”

Watch out. Two things to listen for. First, “**there’s a 7% chance the enemy has 1500 tanks.**” That’s backwards: 7% is how often data like ours would show up *if* 1500 were true. It isn’t the probability that 1500 is true. Correct it now; it’s the most common misreading of a p-value, and they haven’t heard that word yet. Second, “**impossible**” or “**proves.**” 1 in 14 happens. Asking the room where they’d draw the line (5%? 1%?) is a good closer if you have a minute.

Before You Leave

Note. Suggested true value: 897. Nothing in the handout fixes it; choose it before class. 897 is fully consistent with the capture: a fleet of 897 gives a largest serial ≤ 880 in 91% of captures. It also produces the most useful reveal: `est_max` misses by only 17, `est_corrected` by +158 (18%). The biased rule wins this round. A miss of 158 or more happens in about 1 capture in 4 from a fleet of 897 (the corrected rule’s SD there is about 151), so it’s an ordinary miss. If you’d rather the corrected rule look good, 1000 gives +55 and max -120; 950 gives +105 and -70.

— Errors at 897: see the Part 1 table. “Was the method wrong?” No, and the evidence is the Part 3 histogram: a miss this size is typical for one capture, and the method is judged by its distribution, not its one outcome. Being unlucky once is not evidence of a bad method. The honest version of the answer is “we can’t tell from one war; that’s why we simulated a thousand.”

If You Finish Early

E1 They’re a picture of the *procedure*, not of the enemy: every answer the rule would have given across all the captures that could have happened. The one real capture is a single draw from that histogram. That’s how anyone can know how well a method works before the records come in, and it’s why a width can be attached to a single estimate. Name it if you like: a **sampling distribution**.

E2 Both are procedures that are right *on average* (fair in expectation, unbiased) and deliver one outcome from a wide distribution. In Class 10 the spread came from one lumpy item; here it comes from having five tanks. Trusting the procedure means trusting its long-run behavior, which you know from its distribution, and reporting the spread along with the single answer. Strong answers say *what you’d report* (“about 1055, probably within ± 170 ”), which sets up confidence intervals.

Appendix: solution code (instructor only)

Checked in the local ds313 environment. Output of the checked run is in the Part 4 section; your numbers will differ within the class ranges above.

```
def est_corrected(sample):
    return max(sample) * (1 + 1 / len(sample)) - 1

captured = make_array(62, 214, 389, 605, 880)
print(est_max(captured), est_corrected(captured))      # 880  1055.0

corrected_estimates = make_array()
for i in np.arange(1000):
    sample = np.random.choice(serializers, 5, replace=False)
    corrected_estimates = np.append(corrected_estimates, est_corrected(sample))

results.hist('corrected', bins=bins)
plt.title('Estimator: corrected');
```

Part 4 summaries (same pattern as the worked model):

```
print('mean      ', np.round(np.mean(corrected_estimates), 1))
print('median    ', np.round(np.median(corrected_estimates), 1))
print('SD        ', np.round(np.std(corrected_estimates), 1))
print('IQR       ', np.round(np.percentile(corrected_estimates, 75)
                             - np.percentile(corrected_estimates, 25), 1))

close_enough = (corrected_estimates >= 800) & (corrected_estimates <= 1200)
print('within 20%', np.round(np.mean(close_enough), 3))
```

Part 5:

```
claimed = 1500

largest_serials = make_array()
for i in np.arange(1000):
    sample = np.random.choice(np.arange(1, claimed + 1), 5, replace=False)
    largest_serials = np.append(largest_serials, max(sample))

np.mean(largest_serials <= 880)      # about 0.07
```